

Zadanie 4

System Współbieżnego Zarządzania Rozproszoną Siecią Energetyczną

Termin oddania: 12.05 (część I) i 26.05.2025 23:59 CEST (część II)

Maksymalna ocena: 15 punktów

Słownik pojęć

Poniższe nazwy są używane konsekwentnie w całym dokumencie i powinny odpowiadać nazwom typów oraz zmiennych w kodzie.

Nazwa w kodzie	Alternatywne nazwy w treści	Opis
GridHub	Hub, gorutyna centralna	Główna gorutyna balansująca sieć
ESS	Magazyn energii, baterie	Energy Storage System
WeatherStep	Krok pogodowy	5ms realnego czasu = ~5 minut czasu symulacji
GridStep	Krok sieciowy	100ms realnego czasu = 1h czasu symulacji
DemandReport	Żądanie konsumenta	Struktura wysyłana przez konsumenta do GridHub
SupplyStatus	Przydział mocy	Odpowiedź GridHub do konsumenta
SoC	Stan naładowania	State of Charge — poziom naładowania baterii (0.0–1.0)

Założenia symulacji

System używa **dwóch niezależnych skal czasowych**, które odpowiadają dwóm różnym dynamikom fizycznym:

Skala pogodowa (WeatherStep)

```
const WeatherStep = 5 * time.Millisecond // 1 krok = ~5 minut czasu
```

- WeatherStation i Broadcaster działają na tej skali.
- Predictor subskrybuje dane z częstotliwością WeatherStep i buforuje je wewnętrznie.
- Jedna godzina czasu symulacji = **12 kroków WeatherStep** ($12 \times 5\text{min} = 60\text{min}$).
- Jedna pełna doba = **288 kroków WeatherStep** ($288 \times 5\text{min} = 1440\text{min}$).

Skala sieciowa (GridStep)

```
const GridStep = 100 * time.Millisecond // 1 krok = 1 godzina czasu
```

- Decyzje o bilansowaniu sieci, zarządzaniu ESS i load sheddingu podejmowane są co godzinę.
- GridHub, konsumenci i DataLogger działają na tej skali.
- Jedna pełna doba symulacji trwa **2,4 sekundy** realnego czasu ($24\text{ kroki} \times 100\text{ms}$).

Relacja między skalami

Między każdym GridStep a następnym Predictor zbiera **12 odczytów pogodowych** (WeatherStep). Na ich podstawie wylicza trend i wysyła jedną zbiorczą prognozę do GridHub na początku każdego GridStep. To Predictor jest mostem między skalami — tłumaczy gęste dane pogodowe na jedną prognozę godzinową.

Wszystkie interwały podawane w dalszej części zadania (np. „prognoza na 5 kroków w przód”, „szczyt wieczorny o 18:00”) odnoszą się do skali **GridStep** (godzinowej), o ile nie zaznaczono inaczej.

Opis projektu

Celem zadania jest stworzenie symulatora sieci energetycznej w języku Go. System musi zarządzać dynamicznie zmieniającą się produkcją energii z odnawialnych źródeł (zależną od pogody), przewidywać zapotrzebowanie, zarządzać magazynami energii (ESS) oraz kontrolować elektrownie konwencjonalne o wysokiej bezwładności.

Wymagania techniczne

1. **Gorutyny:** Każdy element sieci (elektrownia, odbiorca, stacja pogodowa) musi działać we własnej gorutynie.
 2. **Kanały:** Cała komunikacja między komponentami odbywa się przez kanały. Zakaz używania zmiennych globalnych do przesyłania danych procesowych.
 3. **Interfejsy:** Elementy systemu są implementowane przez interfejsy. Minimalna implementacja musi zawierać:
 - `EnergySource` — definiuje podstawowe zachowania każdego źródła energii,
 - `Predictor` — definiuje zachowania zapewniające analizę danych historycznych w celu przewidywania przyszłej produkcji,
 - `Consumer` — definiuje podstawowe zachowania odbiorcy końcowego energii,
 - `EnergyStorage` — definiuje operacje na magazynach energii (ESS),
 - `WeatherProvider` — definiuje podstawowe zachowania źródła danych pogodowych,
 - `DataLogger` — definiuje funkcjonalności pozwalające na trwały zapis stanu systemu.
 4. **Context:** Pełna obsługa `context.Context` dla mechanizmu `graceful shutdown`.
 5. **Synchronizacja:** Użycie `sync.Mutex` wyłącznie do agregacji statystyk globalnych.
-

Składniki systemu

1. Warunki pogodowe (`WeatherStation`)

`WeatherStation` to niezależna gorutyna symulująca świat zewnętrzny. Działa na skali `WeatherStep` (co 5ms = co ~5 minut czasu symulacji).

Generowanie danych pogodowych: `WeatherStation` nie losuje wartości w oderwaniu od siebie. Przechowuje stan wewnętrzny (np. aktualną prędkość wiatru V_t). W każdym kroku czasowym oblicza:

$$V_{t+1} = V_t + \text{random}(-1, 1)$$

Dzięki temu wykresy produkcji energii są gładkie, co pozwala `Predictor`-owi na analizę trendów.

Przekazywanie danych — `Broadcaster (Pub/Sub)`: Dane pogodowe nie są wysyłane bezpośrednio do farm OZE. Trafiają najpierw do `Broadcaster`-a, który:

- przechowuje listę kanałów wszystkich subskrybentów (farmy OZE, `Predictor`),
- rozgłasza dane do wszystkich subskrybentów z użyciem instrukcji `select` z klauzulą `default`,

- **porzuca** paczkę dla danego subskrybenta, jeśli ten jest zajęty i nie odbiera — dzięki temu wolny subskrybent nie blokuje całej sieci.

```
// Szkielet logiki Broadcastera
for _, ch := range subscribers {
    select {
    case ch <- weatherData:
    default:
        // subskrybent zajęty – porzucamy paczkę dla niego
    }
}
```

2. Warstwa analizy (Predictor)

Predictor pełni rolę inteligencji systemu oraz **mostu między dwiema skalami czasowymi**. Działa jako osobna gorutyna subskrybująca dane z Broadcaster-a z częstotliwością WeatherStep, ale wysyła prognozy do GridHub z częstotliwością GridStep.

Buforowanie danych: Predictor przechowuje wewnątrz slice ostatnich N odczytów pogodowych. Ponieważ jeden GridStep (1h) odpowiada 12 krokom WeatherStep ($12 \times 5\text{min}$), zalecane $N = 12$ — bufor obejmuje wtedy dokładnie jedną godzinę historii pogodowej. Możesz przyjąć większe N dla dłuższej pamięci trendu.

Ekstrapolacja trendu: Po każdym GridStep (nie po każdym odczycie) Predictor wylicza pochodną zmiany na podstawie zebranych odczytów i wysyła do GridHub prognozę:

■ „Prognoza: Za 2 kroków GridStep moc OZE wzrośnie o 15%”

Dzięki gęstym danym z WeatherStep trend jest wyliczany na podstawie 12 punktów pomiarowych na godzinę, a nie 1 — co znacząco poprawia jakość prognozy.

Kanał prognoz: Prognozy płyną dedykowanym kanałem forecastChan, **oddzielnym** od kanału rzeczywistych odczytów mocy z farm OZE. Kanał powinien być buforowany (np. rozmiar 1), aby wolny GridHub nie blokował Predictor-a.

3. Gorutyna centralna (GridHub)

GridHub to najbardziej złożona gorutyna systemu. Działa w pętli select, obsługując cztery główne zdarzenia:

Zdarzenie 1 — Ticker bilansujący (co 1 GridStep = co 1h symulacji): Oblicza różnicę między aktualną produkcją a łącznym popytem zgłoszonym przez konsumentów.

Zdarzenie 2 — Odbiór prognozy z Predictor-a: Porównuje prognozę z planowanym zapotrzebowaniem. Jeśli prognoza przewiduje spadek OZE, GridHub sprawdza status elektrowni węglowej. Jeśli jest Off, wywołuje metodę Start(), uruchamiając procedurę rozgrzewania.

Zdarzenie 3 — Zarządzanie magazynem (ESS):

Stan systemu	Działanie GridHub
Nadwyżka produkcji, SoC < 100%	Wysyła energię do ESS (Charge)
Nadwyżka produkcji, SoC = 100%	Wysyła sygnał do OZE o ograniczeniu mocy (Curtailment)
Niedobór produkcji, SoC > 0%	Rozładowuje ESS (Discharge)
Niedobór produkcji, SoC = 0%	Uruchamia procedurę Load Shedding

Zdarzenie 4 — Obsługa sytuacji awaryjnych (Load Shedding): Gdy bilans jest ujemny, baterie puste, a elektrownia węglowa jeszcze się rozgrzewa, GridHub uruchamia procedurę Load Shedding (patrz sekcja Konsumentci).

4. Konsumentci energii

Każdy odbiorca jest modelowany jako **niezależna gorutyna**, która w każdym kroku GridStep aktualizuje swoje zapotrzebowanie i zgłasza je do GridHub.

Cykl życia konsumenta (każdy krok GridStep)

1. Oblicz aktualne zapotrzebowanie P_demand (na podstawie czasu symu
2. Wyślij DemandReport{ID, P_demand, Priority} do GridHub przez wspo
3. Czeka
4. Jeśli SupplyStatus.AllocatedMW < P_demand → przejdź w stan Partia
5. Zgłoś zdarzenie do modułu statystyk

Typy konsumentów

Odbiorcy domowi (Residential, priorytet 3): Zapotrzebowanie podąża za krzywą dwuszczytową:

- szczyt poranny (kroki odpowiadające ~7:00–9:00)
- szczyt wieczorny (kroki odpowiadające ~18:00–22:00)

Odbiorcy przemysłowi (Industrial, priorytet 2): Wysoki, stabilny pobór w godzinach pracy (kroki ~6:00–18:00) z nagłymi pikami (rozruch ciężkich maszyn). Poza godzinami pracy pobór spada do minimum technologicznego.

Odbiorcy krytyczni (Critical, priorytet 1): Profil niemal płaski (stały pobór przez całą dobę). Najwyższy priorytet dostaw — odłączani jako ostatni lub wcale.

Wymagania implementacyjne

Fan-In (agregacja żądań): Konsumenci nie są odpytywani przez GridHub. To konsumenci **aktywnie wysyłają** swoje DemandReport do jednego, wspólnego kanału demandChan w GridHub.

Dynamiczna rejestracja: System musi pozwalać na dodanie nowego konsumenta (np. nowej fabryki) w trakcie działania symulacji bez restartu.

Algorytm Load Shedding

Gdy $\text{łączna_produkcja} + \text{ESS.Discharge}() < \text{łączny_popyt}$, GridHub wykonuje:

1. Pobierz listę wszystkich aktywnych konsumentów posortowaną według priorytetu (od najniższego: Residential → Industrial → Critical).
2. Odłączaj kolejnych konsumentów (od najniższego priorytetu), aż bilans wróci do zera lub lista się wyczerpie.
3. Wyślij odłączonemu konsumentowi `SupplyStatus{AllocatedMW: 0, Reason: "LoadShed"}`.
4. Stan odłączenia utrzymuje się do następnej iteracji tickera bilansującego — konsument może wrócić do sieci automatycznie, gdy bilans się poprawi.
5. Każde zdarzenie odłączenia jest natychmiast rejestrowane w module statystyk.

5. Archiwizacja danych (DataLogger)

- **Format:** Wszystkie kluczowe metryki systemu muszą być zapisywane do pliku `logs/grid_history.csv` (lub JSON).
 - **Współbieżność:** Zapis jest realizowany przez dedykowanego workera, który odbiera dane asynchronicznie przez kanał — nie blokuje pozostałych gorutyn.
 - **Wydajność:** Użyj `bufio.Writer`, aby uniknąć zbyt częstych operacji dyskowych.
 - **Trwałość:** System musi zagwarantować zapisanie ostatnich danych i wywołanie `Flush` przed zamknięciem aplikacji (obsługa w mechanizmie graceful shutdown).
-

Monitoring i raportowanie

Program wyświetla w konsoli co N kroków GridStep raport stanu w formacie:

```
[Pogoda]    Wiatr: X km/h | Słońce: Y%  
[Produkcja] OZE: A MW | Konwencjonalna: B MW | Baterie: C% (SoC)  
[Sieć]      Popyt: D MW | Bilans: E MW | Stan: [STABLE/CRITICAL]
```

Punktacja (15 pkt)

Komponent	Punkty
Poprawne interfejsy + architektura kanałów	3
WeatherStation + Broadcaster (Pub/Sub)	2
Predictor z ekstrapolacją trendu	2
GridHub (bilans, ESS, Load Shedding)	4
Konsumenci (3 typy, fan-in, dynamiczna rejestracja)	2
DataLogger (CSV/JSON) + graceful shutdown	2

Na co zwrócić szczególną uwagę

- **Graceful Shutdown:** Po Ctrl+C wszystkie gorutyny kończą pracę w kontrolowany sposób (context.Context z anulowaniem). DataLogger jako ostatni wywołuje Flush.
- **System predykcyjny:** Elektrownia węglowa zaczyna rozruch **zanim** nastąpi faktyczny spadek produkcji OZE — Predictor musi wyprzedzać rzeczywistość.
- **Magazyn energii:** ESS nie przyjmuje energii po osiągnięciu SoC = 100%. Nie oddaje energii poniżej SoC = 0%.
- **Deadlock:** System powinien działać stabilnie przez minimum 10 minut symulacji bez zakleszczenia komunikacji.

Terminy oddania

Data	Zakres
12.05	Plan implementacji: interfejsy + architektura kanałów
28.05	Ostateczny termin oddania kompletnego projektu

Dodatkowe wskazówki

- Zaczynij od zdefiniowania interfejsów i struktur komunikacyjnych (DemandReport, SupplyStatus, ForecastReport) — narzucają poprawną architekturę reszty systemu.
- Ustal stałe symulacji na początku pliku i używaj ich konsekwentnie:

```
const WeatherStep = 5 * time.Millisecond // ~5 minut czasu s.  
const GridStep    = 100 * time.Millisecond // 1 godzina czasu  
const WeatherPerGrid = GridStep / WeatherStep // = 12 kroków p  
const ForecastHorizon = 5 // prognoza na 5 kroków GridSte  
const PredictorBufferSize = WeatherPerGrid // bufor = 1 godzi.
```

- Pamiętaj, że Predictor odbiera dane w rytmie WeatherStep, ale wysyła prognozę do GridHub raz na GridStep. Użyj wewnętrznego licznika lub drugiego tickera.
- Użyj select z default w Broadcaster-ze, aby uniknąć blokowania systemu przez wolnych subskrybentów.
- Testuj Load Shedding celowo — np. wyłączając wszystkie farmy OZE i rozładowując baterię do zera.